# Software Requirements Specification

# 0. Document Overview and Conventions

## 0.1 Purpose

This Software Requirements Specification (SRS) defines the functional and non-functional requirements for the Video and Audio Communication, Leaderboard and Data Processing, Calendar Scheduling, Booking, and Notifications and Automated Assessment and Grading . It is the authoritative reference for design, development, testing, and acceptance activities.

## 0.2 Scope

The platform consists of four integrated modules:

- Module 1 – Video and Audio Communication (Interview Room)
- Module 2 – Leaderboard and Data Processing
- Module 3 – Calendar Scheduling, Booking, and Notifications
- Module 4 – Automated Assessment and Grading

## 0.3 Definitions and Acronyms

| Term | Definition |
|---|---|
| RBAC | Role-Based Access Control |
| WebRTC | Web Real-Time Communication |
| DTLS | Datagram Transport Layer Security |
| SRTP | Secure Real-time Transport Protocol |

| | |
|---|---|
| AES-256 | Advanced Encryption Standard, 256-bit key |
| PII | Personally Identifiable Information |
| MVVM | Model-View-ViewModel architectural pattern |
| SRS | Software Requirements Specification |
| UUID | Universally Unique Identifier |
| COMPLETED | Final state of a test attempt after user submission |
| EXPIRED | Final state of a test attempt after the 30-minute timer elapses |
| IN_PROGRESS | Active state of a test attempt currently underway |
| Free | Calendar slot status: available for candidate booking |
| Locked / Occupied | Calendar slot status: already booked by a candidate |

# 1. Video and Audio Communication Module

## 1.1 Module Overview

This module provides an integrated, real-time video and audio interview environment connecting two roles: **Interviewer** (employer representative) and **Interviewee** (job applicant). The interface operates as a secure, dedicated meeting window without requiring external conferencing software.

## 1.2 Stakeholders and Roles

| Role | Description |
|---|---|
| Interviewer | Employer representative; exclusive access to the embedded notes panel during and after the session |
| Interviewee | Job applicant; no access to the notes panel at any time |

## 1.3 Functional Requirements

### 1.3.1 Call Initiation and Session Management

| Req ID | Requirement Statement | Verification Method |
|---|---|---|
| FR-1.1 | Upon initiating or joining a scheduled interview, the system shall render a secure meeting window for both participants within 3.0 seconds on a connection meeting minimum bandwidth criteria (≥15 Mbps download / ≥5 Mbps upload). | Manual Audit via Browser DevTool. Measure time from 'Join' click to the onloadedmetadata event of the video element using a simple console log or the Performance tab. |
| FR-1.2 | The meeting window shall support continuous video rendering and low-latency audio transmission using WebRTC or an equivalent real-time protocol approved by DevSecOps. | Protocol Configuration Check: Inspect the RTCPeerConnection configuration in the source code to ensure iceServers are defined and the protocol is confirmed as WebRTC. Verify "low-latency" by checking the jitterBufferDelay in chrome://webrtc-internals. |
| FR-1.3 | Both participants shall have the following active call controls: (a) mute/unmute microphone; (b) enable/disable camera; (c) disconnect from the call. | Manual + automated UI test – verify each control triggers the correct media state change for both roles. |

| | | |
|---|---|---|
| FR-1.4 | The system shall generate and associate a unique, secure session identifier (UUID) with each interview meeting upon creation. | Unit test – verify UUID uniqueness and correct database association. |

### 1.3.2 Interviewer Notes Panel

| Req ID | Requirement Statement | Verification Method |
|---|---|---|
| FR-1.5 | A dedicated, embedded text editor panel shall be rendered exclusively for the Interviewer role, displayed alongside the live video feed of the Interviewee within the same meeting window. | Role-based UI test – verify panel is present in an Interviewer session and completely absent in an Interviewee session. |
| FR-1.6 | The notes panel shall support alphanumeric text input and basic rich-text formatting: bold, italic, underline, and bullet lists. | Functional test – apply each formatting option and verify correct persistence and retrieval. |

### 1.3.3 Auto-Save and Synchronisation

| Req ID | Requirement Statement | Verification Method |
|---|---|---|
| FR-1.8 | The system shall implement a continuous, non-blocking auto-save mechanism. Notes shall be cached locally and asynchronously synchronised to the backend server at intervals not exceeding 15 seconds. | State Observation Test: Open the application and Network tab. Type notes and verify an XHR/Fetch request triggers every 15s. Manually toggle "Offline" in DevTools, continue typing, toggle "Online," and verify the "Sync" request contains the accumulated text. |
| FR-1.9 | The notes interface shall display a discrete, non-intrusive visual indicator showing: (a) changes are pending sync; (b) changes have been successfully committed to the server. | Manual UI test – verify indicator transitions accurately between states under simulated network conditions. |

### 1.3.4 Session Termination and Final Save

| Req ID | Requirement Statement | Verification Method |
|---|---|---|
| FR-1.11 | Upon explicit termination of the video call by either participant, the system shall intercept the termination event and trigger a final, synchronous save of the Interviewer's notes before the session is destroyed. | Integration test – terminate session and verify final payload is committed before session state is destroyed. |
| FR-1.12 | The system shall verify successful commit of the final note payload to the primary database before permanently destroying the meeting session state. If the commit fails, the system shall retry a configurable number of times before surfacing a recoverable error to the Interviewer. | Mocked Error Simulation: Temporarily point the API endpoint to a non-existent URL or use a tool (like Requestly) to force a 500 error on the final save. Observe the console/network tab to count retry attempts before the UI surfaces the error message. |
| FR-1.13 | Persisted notes shall be automatically mapped and stored relationally to: (a) the Interviewee's user ID; (b) the Interviewer's user ID; (c) the unique job requisition / application identifier. | Database integration test – verify all three foreign key associations are present on every saved notes record. |

### 1.3.5 Interview Session Generation

| Req ID | Requirement Statement | Verification Method |
|---|---|---|
| FR-3.19 | Upon completion of a slot booking, the system shall automatically generate an InterviewSession object containing: session_id, slot_id, candidate_id, acces_link_hash, notes_encrypted, status. | Integration test – complete a booking; verify InterviewSession record is created with all required fields populated. |

| | | |
|---|---|---|
| FR-3.20 | The system shall generate a unique, secure participation link for each InterviewSession. Accessing the link before the scheduled start time shall display a waiting message. Access without authentication shall be denied. Access to the link after the reserved interview period has ended shall also be denied. | Functional test – access link (a) before start time: verify waiting message; (b) unauthenticated: verify access denied; (c) at scheduled time: verify session opens. (d) after the reserved interview period has ended: verify access is denied. |

## 1.4 Non-Functional Requirements

### 1.4.1 Performance

| Req ID | Requirement Statement | Verification Method |
|---|---|---|
| NFR-1.1 | The initial connection and first rendered video frame shall be established within 3.0 seconds on standard broadband (≥15 Mbps down / ≥5 Mbps up). | Automated performance test – measure end-to-end connection time from initiation to first video frame. |
| NFR-1.2 | In the event of bandwidth degradation below the minimum threshold, the system shall automatically downscale video resolution, ultimately disabling video to maintain uninterrupted audio continuity. | Manual Throttle Test: While in an active call, use the Chrome "Network" tab to switch the profile from "Online" to "Slow 3G." Verify that the video freezes or disappears while audio remains audible (via local loopback or second device). |

### 1.4.2 Security

| Req ID | Requirement Statement | Verification Method |
|---|---|---|
| NFR-1.3 | All active video and audio media streams shall be encrypted in transit using DTLS and SRTP. | Security audit – verify DTLS/SRTP headers via network packet capture during an active session. |

| | | |
|---|---|---|
| NFR-1.4 | Interviewer notes shall be encrypted at rest in the database using AES-256 or higher. | Security audit – verify encryption key management and column-level encryption configuration. |
| NFR-1.5 | Post-interview access to saved notes shall be restricted to: (a) the original Interviewer; (b) explicitly authorised organisational administrators; (c) authorised hiring managers. All other roles shall be denied access. | RBAC access matrix test – attempt access with each platform role and verify correct allow/deny outcomes. |

# 2. Leaderboard and Data Processing Module

## 2.1 Module Overview

This module ranks candidates based solely on their performance in platform-administered automated assessments. Recruiters and administrators use the leaderboard view to evaluate relative candidate performance. After a candidate finalises a test attempt, the system recalculates the leaderboard and displays a summary leaderboard showing the top three ranked candidates together with the candidate's own ranking position. Users may access the complete leaderboard through a dedicated view.

## 2.2 Data Sources

| Table | Role in This Module |
|---|---|

| | |
|---|---|
| Tests | Stores test definitions, questions, correct answers, and per-question scores. Used to determine maximum possible score for normalisation. |
| UserTest | Stores per-user test attempts, statuses, raw scores, and completion timestamps. Primary data source for all ranking computations. |

## 2.3 Functional Requirements

### 2.3.1 Data Ingestion and Validation

| Req ID | Requirement Statement | Verification Method |
|---|---|---|
| FR-2.1 | After a test attempt is finalised, the system shall extract from UserTest: external_user_id, test_id, score, completed_at, and status. | Unit test – verify all five extracted fields match the database record exactly. |
| FR-2.2 | Before processing, the system shall validate: (a) the candidate exists; (b) the test_id corresponds to a valid Tests record; (c) grading has been successfully completed. | Unit test – submit records with invalid candidate ID, invalid test_id, and incomplete grading status; verify rejection in each case. |
| FR-2.3 | Only attempts with status COMPLETED or EXPIRED shall proceed to ranking computation. All other statuses shall be rejected. | Unit test – submit an IN_PROGRESS record; verify rejection and leaderboard unchanged. |
| FR-2.4 | Invalid or incomplete submissions shall be rejected and logged with: record identifiers, rejection reason, and timestamp. Rejected records shall not affect rankings. | Integration test – inject an invalid record; verify leaderboard unchanged and error log entry created. |

### 2.3.2 Score Normalisation

| Req ID | Requirement Statement | Verification Method |
|---|---|---|

| | | |
|---|---|---|
| FR-2.5 | Raw scores shall be converted to percentage-based normalised scores prior to ranking. Formula: Normalised Score = (Raw Score ÷ Maximum Possible Score) × 100. | Unit test – verify formula output for at least three tests with differing maximum scores. |
| FR-2.6 | The maximum possible score for a test shall be computed dynamically by summing the question_score values of all questions sharing that test_id in the Tests table. It shall not be a static field. | Unit test – verify sum computation against a manually calculated fixture for at least two tests. |

### 2.3.3 Ranking Computation

| Req ID | Requirement Statement | Verification Method |
|---|---|---|
| FR-2.7 | The leaderboard shall support two ranking scopes: (a) Single-test ranking – candidates ordered by normalised score for one selected test; (b) Aggregated ranking – composite score across all completed tests per candidate. | Functional test – verify both scopes return the correct candidate order for a known dataset. |
| FR-2.8 | In the aggregated ranking, the composite score shall be computed by applying administrator-configurable weight coefficients to each normalised test score before summation. Formula: Composite Score = Σ (Normalised Score × Weight Coefficient). | Unit test – verify composite score against manually calculated results for known weights and normalised scores. |
| FR-2.9 | Weight coefficients shall be configurable by administrators via the admin interface and persisted in the system configuration layer. They shall not be hard-coded. | Admin UI test – update a weight coefficient; verify leaderboard recalculates and reflects the new weighting. |

| | | |
|---|---|---|
| FR-2.10 | Tie-breaking for identical final scores shall be applied in this deterministic order: (1) higher average normalised score across all completed tests; (2) earliest completed_at timestamp. | Unit test – construct a tie scenario and verify identical ordering across at least 10 repeated computations. |

### 2.3.4 Leaderboard Display and Access Control

| Req ID | Requirement Statement | Verification Method |
|---|---|---|
| FR-2.11 | After a candidate finalises a test attempt and grading is completed, the system shall display a summary leaderboard containing the top three ranked candidates for that assessment, ordered by their computed ranking position. The interface shall also display the candidate's own ranking position and provide a "See Full Leaderboard" option allowing access to the complete leaderboard view. | Manual UI test – verify that the top three ranked candidates and the candidate's own ranking position are displayed after test completion and that the "See Full Leaderboard" option opens the full leaderboard view. |
| FR-2.12 | Candidates may view their own performance details and ranking position, together with a limited summary leaderboard displaying the top three ranked candidates. Recruiters and Administrators may view the complete leaderboard rankings and detailed score breakdowns. | RBAC access test – verify API response filters data correctly for each role. |
| FR-2.13 | The leaderboard shall recalculate when: (a) new validated results are inserted; (b) a test attempt transitions to COMPLETED or EXPIRED; (c) the leaderboard view is accessed. | Integration test – trigger each recalculation event independently; verify rankings update after each. |

### 2.3.5 Architecture

| Req ID | Requirement Statement | Verification Method |
|---|---|---|
| FR-2.14 | The module shall follow the MVVM pattern: assessment entities in the Model layer (Tests, UserTest); validation, normalisation, and ranking logic in the Service layer; presentation in ViewModels and Views. | Code review – verify layering compliance; confirm no business logic exists in View or ViewModel layers. |
| FR-2.15 | All ranking computations shall rely exclusively on data stored in the relational database, ensuring deterministic and reproducible outcomes for any given dataset. | Regression test – run ranking computation on the same dataset twice; verify bit-identical results. |

# 3. Calendar Scheduling, Booking, and Notifications Module

## 3.1 Module Overview

This module serves as the dedicated scheduling engine for the platform, taking over the workflow once a candidate's application is approved by a company. It manages the end-to-end scheduling lifecycle: recruiters define their calendar availability, eligible candidates book interview slots specific to their assigned recruiters, and the system automatically handles the transactional locking, session generation, and automated notifications.

## 3.2 Stakeholders and Roles

| Role | Responsibilities |
|---|---|
| Recruiter | Defines and manages availability slots; views the monthly calendar; configures slot details. |
| Candidate | Accesses the Interviews Dashboard to view companies that accepted their application; clicks the 'Schedule Interview' button to browse available slots for |

| | |
|---|---|
| | a specific company; books one slot per invitation. |
| Platform Admin | May view all calendars; can override slot statuses in exceptional circumstances. |
| Notification Engine | Internal system component dispatching automated in-app notifications at defined lifecycle events. |

## 3.3 Functional Requirements

### 3.3.1 Eligibility and Access Control

| Req ID | Requirement Statement | Verification Method |
|---|---|---|
| FR-3.1 | Access to the Calendar Module shall be controlled by the ApplicationState table. The module shall automatically become accessible to a candidate as soon as their ApplicationState record is updated to interview-eligible status. No manual intervention shall be required. | Integration test – update ApplicationState to eligible; verify Calendar Module unlocks without manual action. |
| FR-3.2 | A candidate shall only be able to view and interact with slots created by the recruiter assigned to their specific application. Slots belonging to other recruiters shall not be visible or accessible. | RBAC integration test – verify a candidate cannot retrieve or book slots assigned to a different recruiter via any API call. |

### 3.3.2 Recruiter Availability Management

| Req ID | Requirement Statement | Verification Method |
|---|---|---|
| FR-3.3 | The recruiter interface shall provide a monthly calendar grid. Clicking a specific day shall | Manual UI test – verify monthly grid renders and day selection opens the correct |

| | | |
|---|---|---|
| | open the day-management view for that date. | management area. |
| FR-3.4 | Slot creation shall be initiated by clicking a time cell, which opens a "Create Slot" configuration window pre-populated with the selected date. | Manual UI test – verify the configuration window opens with the correct date pre-filled. |
| FR-3.5 | Each slot shall be defined by: (a) Start Time; (b) End Time; (c) Duration selected from a fixed menu of 30 or 60 minutes. | Unit test – create slots with each duration option and verify all three parameters stored correctly. |
| FR-3.6 | The system shall enforce slot alignment: all slot start times must align to the hour or half-hour mark (e.g. 09:00, 09:30). Non-aligned slots shall be rejected at the server with a user-facing validation error. | Unit test – attempt to create slots with misaligned start times; verify all are rejected with the correct error message. |
| FR-3.7 | Slots with status "Free" shall be fully editable and deletable by the owning recruiter. Slots with status "Locked" shall be read-only and cannot be modified or deleted. | Functional test – attempt to edit and delete a Locked slot via both UI and API; verify both operations are blocked. |
| FR-3.8 | The monthly calendar grid shall use a colour-coded heatmap to indicate slot occupancy levels at a glance. | Manual UI test – verify colour coding accurately reflects occupancy data for days with zero, partial, and full bookings. |

### 3.3.3 Candidate Booking Interface

| Req ID | Requirement Statement | Verification Method |
|---|---|---|
| FR-3.9 | The candidate interface shall provide a horizontal date-picker bar enabling navigation between available weekdays. Upon selecting a date, the system shall populate an interactive grid with time cards representing exclusively | Manual UI test – select multiple dates and verify the slot grid updates correctly for each. |

| | | |
|---|---|---|
| | 'Free' slots, displayed in chronological order. | |
| FR-3.10 | The candidate shall only see slots with status 'Free' assigned by the recruiter of the currently selected company. Past slots, booked slots, and slots from other recruiters (including recruiters from other companies the candidate is accepted to) shall not be displayed. | Integration test – verify the API response excludes non-free, past, and unassigned slots, and strictly returns only the slots belonging to the recruiter associated with the specific company selected from the dashboard. |
| FR-3.11 | Available time cards shall be displayed as interactive time cards in chronological order, each showing: start time and end time. | Manual UI test – verify display format, chronological ordering, and contextual field rendering. |
| FR-3.12 | Selecting a time card shall visually highlight it to provide clear selection feedback before confirmation. | Manual UI test – verify highlight state is applied on selection and cleared on deselection. |

### 3.3.4 Transactional Booking Logic

| Req ID | Requirement Statement | Verification Method |
|---|---|---|
| FR-3.13 | Upon booking confirmation, the system shall execute the following atomic three-step process: (1) query the database in real time to confirm the slot is still "Free"; (2) if Free, atomically update status to "Occupied" and associate it with the candidate's profile; (3) return a success confirmation. All three steps shall occur within a single database transaction. | Integration test – simulate two candidates booking the same slot concurrently; verify exactly one succeeds and the other receives an error. |
| FR-3.14 | If the selected slot is no longer available at confirmation time, the system shall: (a) block the booking; (b) display "This slot is no longer available"; (c) force a refresh of the candidate's slot list. | Integration test – race condition simulation; verify correct error message and list refresh after a failed attempt. |

| | | |
|---|---|---|
| FR-3.15 | Upon a successful booking, the slot shall immediately become invisible to all other candidates. It shall not appear in any subsequent API responses for Free slots. | Integration test – book a slot with one account; verify it no longer appears in a second candidate's slot list querying simultaneously. |

### 3.3.5 Automated Notification Engine

| Req ID | Requirement Statement | Verification Method |
|---|---|---|
| FR-3.16 | Upon candidate enrollment confirmation, the system shall automatically dispatch: (a) a confirmation message to the candidate; (b) an alert to the recruiter. Both notifications are sent in-app without manual action. | Integration test – complete enrollment flow; verify both recipients receive email and in-app notification within 60 seconds. |
| FR-3.17 | Immediately upon successful slot booking, the system shall dispatch meeting details (date, time) to both candidate and recruiter through an in-app message. | Integration test – complete a booking; verify both parties receive the confirmation notification with correct details within 60 seconds. |
| FR-3.18 | The system shall automatically send a reminder notification to both parties 24 hours before the scheduled interview start time through in-app messages. | Integration test – schedule an interview 25 hours ahead; verify reminder dispatched within a 5-minute window of the 24-hour mark. |

# 4. Automated Assessment and Grading Module

## 4.1 Module Overview

This module provides a fully automated, deterministic, and reproducible assessment platform for standardised computer science tests. All grading occurs server-side without manual intervention and produces auditable, reproducible results.

**Global rules enforced across all assessments:**

- Each test session has a maximum duration of 30 minutes

- A user cannot retake a test with status COMPLETED or EXPIRED
- The system automatically assigns the next available test in the same category upon completion
- All grading and test management operations rely solely on the Tests and UserTest tables

## 4.2 Data Model

### 4.2.1 Tests Table

Stores test definitions and all questions. Each question is a separate row sharing the same test_id.

| Column | Type and Description |
|---|---|
| test_id | INT / UUID – Primary Key |
| title | NVARCHAR – human-readable test title |
| category | VARCHAR – grouping used for automatic test replacement |
| question_id | INT – question identifier within the test |
| question_text | NVARCHAR – question text presented to the user |
| question_type | VARCHAR – MULTIPLE_CHOICE\|SINGLE_CHOICE \| TRUE_FALSE \| TEXT |
| question_score | DECIMAL – points awarded for a correct answer |
| question_answer | NVARCHAR – correct answer for server-side evaluation only; never exposed to clients |

### 4.2.2 UserTest Table

Stores one record per user per test attempt. Enforces retake prevention and tracks grading state.

| Column | Type and Description |
|---|---|
| userTest_id | INT / UUID – Primary Key |
| test_id | INT – Foreign Key referencing Tests.test_id |
| external_user_id | VARCHAR – platform user identifier |
| score | DECIMAL – computed and written server-side only; not writable by client |
| status | VARCHAR – IN_PROGRESS \| COMPLETED \| EXPIRED |
| completed_at | DATETIME – UTC timestamp of finalisation; null while IN_PROGRESS |

## 4.3 Functional Requirements

### 4.3.1 Test Initialisation

| Req ID | Requirement Statement | Verification Method |
|---|---|---|
| FR-4.1 | When a user attempts to start a test, the system shall query UserTest using external_user_id and test_id. If a record exists with status COMPLETED or EXPIRED, access shall be denied and a descriptive message displayed. | Unit test – attempt to start a test with an existing COMPLETED record; verify denial and record unchanged. |
| FR-4.2 | If no COMPLETED or EXPIRED attempt exists, the system shall create a new UserTest record with status IN_PROGRESS and immediately start the 30-minute server-side | Unit test – start a new test attempt; verify record created with correct initial fields and timer starts immediately. |

| | | |
|---|---|---|
| | session timer. | |

### 4.3.2 Timer Enforcement

| Req ID | Requirement Statement | Verification Method |
|---|---|---|
| FR-4.3 | Each test session shall have a maximum duration of 1800 seconds (30 minutes). The timer shall be enforced exclusively server-side. The client-side timer is for display purposes only and shall not trigger finalisation. | Integration test – allow the server-side timer to expire without any client submission; verify automatic finalisation. |
| FR-4.4 | If the 30-minute limit expires before submission, the system shall automatically finalise the attempt: set status to EXPIRED and compute the score based only on answers submitted up to that point. | Integration test – submit partial answers then allow expiry; verify EXPIRED status and score reflects submitted answers only. |
| FR-4.5 | After a test attempt is finalised, no modification to submitted answers shall be permitted. Any client attempt to modify answers on a COMPLETED or EXPIRED record shall be rejected by the server. | Security test – send an API request to modify an answer on a finalised record; verify rejection with an appropriate error code. |

### 4.3.3 Grading Logic

Each SINGLE_CHOICE and MULTIPLE_CHOICE question has 6 possible answers.
MULTIPLE_CHOICE questions must have at least 1 correct answer and at most 6 correct answers..

| Req ID | Requirement Statement | Verification Method |
|---|---|---|
| FR-4.6 | When a test is submitted, the system shall evaluate all questions belonging to that test_id by comparing each submitted answer against the corresponding question_answer value in the Tests table. | Unit test – submit known correct and incorrect answers; verify evaluation correctly identifies matches and mismatches. |

| | | |
|---|---|---|
| FR-4.7 | For each question: if the submitted answer exactly matches question_answer, the user receives the full question_score. For single choice and true or false question types, 0 points are awarded in case the answer does not exactly match question_answer. For multiple choice there is a partial score computed using the formula: **Score = MaxScore * max(0, SelectedCorrectAnswers / TotalCorrectAnswers - SelectedIncorrectAnswers / (TotalOptions - TotalCorrectAnswers)).** For the short text question types, 0 points are awarded in case the answer does not exactly match question_answer. | Unit test – verify binary scoring logic for each of the four question types. |
| FR-4.8 | **Total Score = Σ question_score** for all questions where submitted answer matches question_answer. | Unit test – verify total score against manually calculated values for a fixture dataset of at least 10 questions. |
| FR-4.9 | After grading, the system shall update the UserTest record in a single atomic database transaction: (a) update the score field; (b) set status to COMPLETED; (c) set completed_at to current UTC timestamp. | Integration test – verify all three fields updated atomically; inject database failure mid-transaction and verify complete rollback. |

### 4.3.4 Supported Question Types

| Req ID | Requirement Statement | Verification Method |
|---|---|---|
| FR-4.10 | MULTIPLE_CHOICE: User selects one option. If the selected option exactly matches question_answer, award full question_score. Otherwise, compute partial score. If the score is < 0 then it will be set to 0. | Unit test – verify correct and incorrect selection for multiple questions. |
| FR-4.11 | TRUE_FALSE: User selects True or False. If the value exactly matches question_answer, award full question_score. Otherwise, 0. | Unit test – verify both correct and incorrect True/False selections. |

| | | |
|---|---|---|
| FR-4.12 | TEXT: User provides free-text input. Matching rules (case-sensitivity, whitespace trimming) shall be defined per OQ-4.1. If the values match under the defined rules, award full question_score. Otherwise, 0. | Unit test – verify exact match, near-match (wrong case), and incorrect answer scenarios. |
| FR-4.13 | SINGLE_CHOICE: User selects one option. If the selected option exactly matches question_answer, award full question_score. Otherwise, award 0. | Unit test – verify both correct and incorrect selection for single choice. |

### 4.3.5 Automatic Test Replacement

| Req ID | Requirement Statement | Verification Method |
|---|---|---|
| FR-4.14 | After a test is finalised, the system shall: (1) retrieve the category of the completed test; (2) search for another test_id in the same category where the user has no COMPLETED or EXPIRED record; (3) if found, make it available to the user. | Integration test – complete a test; verify the next test in the same category becomes available in the user's test list. |
| FR-4.15 | If no additional untaken test exists in the same category, the system shall display: "No additional tests available in this category." | Functional test – exhaust all tests in a category; verify the correct message is displayed and no error state occurs. |

## 4.4 Security Constraints

| Req ID | Requirement Statement | Verification Method |
|---|---|---|
| NFR-4.1 | Score calculation shall occur exclusively server-side. The server shall not accept any score value originating from the client. | Security test – attempt to POST a score to the UserTest endpoint; verify it is ignored and server computes independently. |

| | | |
|---|---|---|
| NFR-4.2 | Users shall not be able to modify submitted answers after the attempt is finalised. | Security test – attempt to modify an answer on a COMPLETED or EXPIRED record; verify rejection. |
| NFR-4.3 | Users shall not be able to initiate a new attempt for a test with an existing COMPLETED or EXPIRED record. | Unit test – attempt retake after both COMPLETED and EXPIRED; verify denial in both cases. |
| NFR-4.4 | Direct modification of the score field from the client shall not be permitted at any API layer. The score field shall be writable only by the internal grading service. | Security/penetration test – attempt direct score modification via each API endpoint; verify all attempts rejected. |
| NFR-4.5 | The question_answer field shall not be included in any API response sent to a client during an active test session. | Security test – inspect all API responses during an active test session; verify question_answer is absent from every payload. |

## 4.5 Edge Case Handling

| Scenario | System Behaviour | Test Condition |
|---|---|---|
| User submits with no answers | Score = 0. Status set to COMPLETED. completed_at recorded. | Unit test – submit empty answer set. |
| 30-minute timer expires before submission | Status set to EXPIRED. Score computed on answers submitted up to expiry. No further answers accepted. | Integration test – allow timer to expire with partial answers. |
| User attempts to retake a COMPLETED test | Access denied. Existing record unchanged. User-facing message displayed. | Unit test – attempt retake after COMPLETED record exists. |

| | | |
|---|---|---|
| User attempts to retake an EXPIRED test | Access denied. Same behaviour as COMPLETED retake. | Unit test – attempt retake after EXPIRED record exists. |
| Submitted answer format does not match question type | Treated as incorrect. Score = 0 for that question. No error raised to user. | Unit test – submit invalid format for each of the three question types. |
| No additional test in category | System displays: "No additional tests available in this category." | Integration test – exhaust all tests in a category. |
| Database unavailable during grading | Transaction rolled back. Status remains IN_PROGRESS. User notified to retry. | Fault injection test – cut DB connection during submission. |

# 5. Cross-Module Constraints and Integration Requirements

## 5.1 Shared Authentication and Identity

| Req ID | Requirement Statement | Verification Method |
|---|---|---|
| CR-5.1 | All modules shall rely on the same authenticated user identity (external_user_id) as the single authoritative identifier. A user must be authenticated before accessing any module endpoint. | Integration test – send an unauthenticated request to at least one endpoint per module; verify 401 response in every case. |

| | | |
|---|---|---|
| CR-5.2 | Role assignments (Interviewer, Interviewee, Recruiter, Candidate, Admin) shall be managed centrally by the authentication service and consumed by each module's RBAC logic. No module shall maintain its own separate role store. | Integration test – change a user's role in the central auth service; verify all module permissions reflect the updated role immediately. |

## 5.2 Module Integration Points

| Integration | Description |
|---|---|
| Module 3 → Module 1 | A completed slot booking in Module 3 triggers InterviewSession creation, which provides the unique access URL consumed by Module 1 to open the meeting room. |
| Module 4 → Module 2 | Module 4 finalises test attempts (COMPLETED/EXPIRED). Module 2 ingests these records to trigger leaderboard recalculation and displays a summary leaderboard for the completed assessment, together with the candidate's own ranking position. |
| All Modules → Auth Service | All modules consume the central authentication token and RBAC service for identity verification and permission enforcement. |
| Module 3 → Notification Engine | Booking and enrollment events in Module 3 trigger automated notifications dispatched by the Notification Engine. |

## 5.3 Audit and Logging

| Req ID | Requirement Statement | Verification Method |
|---|---|---|

| | | |
|---|---|---|
| CR-5.3 | All write operations across all modules (note saves, leaderboard recalculations, slot bookings, test submissions) shall be recorded in an immutable audit log containing: actor user ID, operation type, affected record identifier, and UTC timestamp. | Integration test – perform one write operation per module; verify a correctly structured audit log entry is created for each. |